

# Semantic Segmentation Competition

**Subject:** Computer Vision, **Year:** 2025

**Student Name:** Kshitij Desai, Sakshi Vishwakarma

**Student ID:** a1936769, a1934902

## 1 Introduction

### 1.1 Task Description

Semantic segmentation is a dense pixel-level classification across an entire image where a class label is assigned to every pixel, this creates detailed understanding of scene composition [1]. The dataset consists up of 150 training images and 50 testing images from various driving scenarios, with pixel-level annotations for 19 semantic classes including road, sidewalk, building, wall etc. The challenge is to optimize segmentation accuracy with computational efficiency, which is to maximize mean Intersection over Union (mIoU) while minimizing the computational cost measured in GFLOPs. The evaluation metric in this challenge is mean IoU, here IoU for each class equals the intersection area divided by the union area between the predicted and the truth regions [2]. This metric captures object detection accuracy and boundary localization precision.

### 1.2 Baseline Model Architecture Discussion

The baseline model is a traditional Fully Convolutional Network (FCN) architecture following an encoder-decoder paradigm. Here the encoder consists of five convolutional stages with progressive channel expansion from 3 input channels to 2048 feature channels. Each stage has convolutions followed by ReLU activation function and max pooling for downsampling. The decoder on the other hand, utilizes three transposed convolution layers to restore resolution from 1/16 back to full resolution, with final **bilinear interpolation** which ensures the exact size matching.

### 1.3 Baseline Model Limitations

- The baseline uses basic Adam optimization without learning rate scheduling, minimal data augmentation, and evaluation only after epoch 100, which may limit the performance potential.
- The model consumes 66.97 GFLOPs, this may happen due to channel expansion from 512 to 2048 in the **stage-5** encoder stage. The expansion with 3×3 convolutions might have created computational congestion.
- The usage of max pooling operations discards spatial information which is crucial for boundary localization, the 1/16 resolution loses fine details that which can't be discovered during upsampling.

## 2 Final Modified Model Discussion

### 2.1 Architecture Selection

We modified the traditional FCN baseline and achieved structure like FastSCNN (Fast Segmentation Convolutional Neural Network), after experimenting with various easy to complex architectures. Final model is a dual-branch design that processes spatial and semantic information through separate specialized pathways rather than a single computationally expensive encoder.

### 2.2 Detailed Architecture Components

- **Learning to Downsample Module:** This part reduces input resolution from full size to 1/8 while preserving spatial information. The design uses one standard convolution followed by two depthwise separable convolutions, creating a progres-

sive resolution reduction pathway: Full then 1/2 then 1/4 then 1/8. Channel progression follows 3 then 32 then 48 then 64, maintaining computational efficiency.

- **Global Feature Extractor:** The model works on 1/8 resolution output from the last module, and reduces to 1/32 while extracting important semantic features[5]. There are 3 blocks of inverted residual layers with channel progression 64 to 96 to 128. Then a global context branch using **adaptive average pooling** is used [4].
- **Feature Fusion Module:** We combine high-resolution features (1/8 resolution, 64 channels) with low-resolution features (1/32 resolution, 128 channels). The module first up-samples semantic features to match spatial using **bilinear interpolation**, then applies 1×1 convolutions to align channel dimensions before **element-wise addition**.
- **Classifier Module:** The final component combines features into class predictions through two **depthwise separable convolutions**, **dropout regularization to prevent overfitting**, and a final 1×1 convolution mapping to 19 output classes.

### 2.3 Efficient Building Blocks

- **Depthwise Separable Convolutions:** We use this technique to separate spatial filtering (depthwise convolution where each input channel is convolved with its own filter) from channel mixing (pointwise 1×1 convolution). This separation extensively reduces parameters and computations.
- **Inverted Residual Blocks:** Inspired by MobileNetV2, inverted residuals first expand channels for better representations [3] followed by depthwise convolution for spatial processing, then project to lower dimensions.

### 2.4 Advanced Training Methods

- **Mixed Precision Training:** We implemented an automatic mixed precision to reduce memory usage and accelerates training enabling larger batch sizes and faster convergence without sacrificing accuracy.
- **Combined Loss Function:** Developed a hybrid loss combining **Cross-Entropy** and **Dice Loss** in an **0.8:0.2 ratio** where cross-Entropy Loss handles pixel classification while Dice Loss specifically targets segmentation quality and addresses class imbalance.
- **Data Augmentation:** Implemented extensive augmentation strategies (**after lots of trial and error**) [6] including multi-scale training with random scaling factors (0.75 times, 1.0 times, 1.25 times), random horizontal flipping, color jittering across brightness, contrast, and saturation dimensions, Gaussian blur application, and gamma correction.
- **Optimization Strategy:** After extensive trials and errors and various parameter values we used **AdamW optimizer with weight decay regularization, cosine annealing learning rate scheduling for smooth convergence, gradient clipping for training stability, and early stopping with patience**.

### 2.5 Ablation Study

There were numerous rounds of trial and error throughout. We began by modifying the layers of the baseline model (SegNet)

| Category          | Baseline                          | Modified Final Model                                                            |
|-------------------|-----------------------------------|---------------------------------------------------------------------------------|
| Architecture      | Simple encoder-decoder (5 stages) | Dual-branch: Learning to Downsample + Global Feature Extractor + Feature Fusion |
| Building Blocks   | Standard convolutions + maxpool   | Depthwise separable convolutions + inverted residuals                           |
| Learning Rate     | 3e-4 (static)                     | 2e-3 (cosine annealing to 1e-6)                                                 |
| Batch Size        | 4                                 | 16                                                                              |
| Optimizer         | Adam                              | AdamW (weight_decay=1e-2, grad_clip=1.0)                                        |
| Loss Function     | CrossEntropyLoss only             | 0.8×CrossEntropyLoss + 0.2×DiceLoss                                             |
| Data Augmentation | Resize + normalize only           | Multi-scale + horizontal flip + color jitter + blur + gamma correction          |
| Training Features | Basic training loop               | Mixed precision + early stopping (patience=25) + model checkpointing            |
| Memory Management | inplace=True operations           | inplace=False + torch.cuda.empty_cache() + gc.collect()                         |
| Evaluation        | After epoch 100 only              | Every 5 epochs with comprehensive monitoring                                    |

**Table 1:** Baseline vs Final Model: Key Implementation Differences

| Method                      | mIoU  | GFLOPs | Efficiency |
|-----------------------------|-------|--------|------------|
| Baseline (FCN- Given Model) | 0.284 | 66.97  | 0.42       |
| Baseline Seg Former         | 0.689 | 15.52  | 4.44       |
| SegFormer Pruned + KD       | 0.552 | 7.35   | 7.52       |
| Fast-SCNN (Final Model)     | 0.445 | 2.17   | 20.54      |

**Table 2:** Ablation Study Table

and tuning its hyperparameters. Then, we experimented with different architectures and carefully tuned their parameters. Since it is difficult to capture every single experiment due to page limits, here we only present the main methods (refer Table 2) that gave meaningful improvements. For the complete ablation study and detailed experimentation, please refer to the backup and experimentation files submitted in the accompanying zip folder.

### 3 Optimizing Cost-Performance Computation

#### 3.1 Architectural Efficiency Strategy

1. The main approach to reduce computational cost is the dual-branch architectural design that separates spatial and semantic processing rather than using a single expensive encoder decoder pipeline.
2. The **Learning to Downsample** module processes spatial information at lower resolutions (Full to 1/8) using efficient **depthwise separable convolutions**. This approach preserves boundary information avoiding the computational overhead of processing high resolution features.
3. We have **Global Feature Extractor** which operates totally at low resolution (1/8 to 1/32), dramatically reducing computational requirements for semantic feature extraction.
4. The global context branch uses **adaptive pooling** to capture scene-level information efficiently without expensive full-resolution processing.

#### 3.2 Memory and Parameter Optimization

1. FastSCNN maintains maximum channel dimensions of 128 compared to 2048 in the baseline, directly reducing memory footprint and computational requirements.
2. There is total parameter is 1.58 million in FastSCNN which is a extreme reduction compared to baseline results, this have happened due to our **depthwise separable convolutions and elimination of expensive fully connected layers**.

### 3.3 Accuracy Efficiency Trade-Off

1. The key to maintaining accuracy while reducing computation lies in the intelligent **feature fusion innovation module** Which combines high-resolution spatial details with low-resolution semantic context and preserves accuracy.
2. Unlike single-scale processing in the baseline, FastSCNN preserves the information simultaneously at multiple resolutions. The spatial branch preserves fine details while the semantic branch captures global context, with fusion combining both perspectives effectively.
3. Linear bottlenecks in **inverted residual blocks** prevent information loss during dimension changes.

## 4 Limitations and Conclusion

### 4.1 Current Limitations

- **Dataset Constraints:** Training and Testing on only 150 and 50 images limits generalization and provides limited statistical confidence for broader deployment.
- **Architectural Simplifications:** The feature fusion use basic element-wise addition rather than any learned attention mechanisms.
- **Evaluation Scope:** Single mIoU metric doesn’t capture boundary accuracy, class-specific performance, or temporal consistency crucial for segmentation in general.
- **Hardware Constraints:** While FastSCNN demonstrates good efficiency on GPU architectures, the depthwise separable convolutions may not achieve acceleration on all hardware platforms.

### 4.2 Conclusion

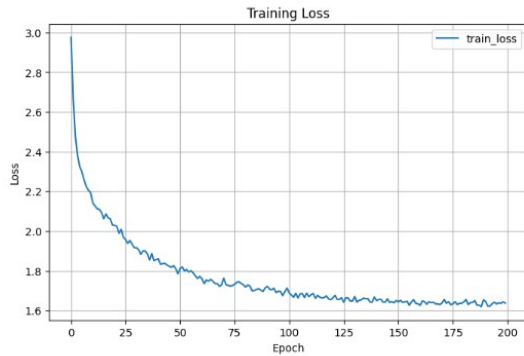
Our work concludes that architectural innovation can achieve superior accuracy-efficiency trade-offs compared to traditional approaches. Moving from a traditional style baseline (66.97 GFLOPs) to FastSCNN, with key Contributions including Dual-branch processing, depthwise separable convolutions, inverted residual blocks, and an advanced training pipeline with mixed precision, combined losses, and extensive augmentation. Our model delivers 44.58% accuracy at 2.17 GFLOPs, enabling real-time deployment and also demonstrates that efficiency and accuracy can coexist in computer vision models.

## References

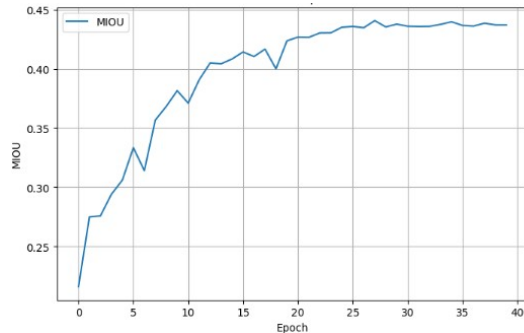
- [1] A. Bosco, “An introduction to Semantic Segmentation,” Aug. 24, 2023. [Online]. Available: <https://medium.com/data-reply-it-datatech/an-introduction-to-semantic-segmentation-4a01f68cf2a3>
- [2] G. Boesch, “What is Intersection over Union (IoU)?,” Jan. 4, 2024. [Online]. Available: <https://viso.ai/computer-vision/intersection-over-union-iou/>
- [3] P.-L. Pröve, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” Apr. 11, 2018. [Online]. Available: <https://medium.com/data-science/mobilenetv2-inverted-residuals-and-linear-bottlenecks-8a4362f4ffd5>
- [4] A. K. Pandey, “Adaptive Average Pooling Layer,” Apr. 13, 2024. [Online]. Available: <https://medium.com/@akp83540/adaptive-average-pooling-layer-cb438d029022>
- [5] S. Shao, K. Chen, A. Karpur, Q. Cui, A. Araujo, and B. Cao, “Global Features are All You Need for Image Retrieval and Reranking,” *arXiv:2308.06954 [cs.CV]*, Aug. 14, 2023, [Online]. Available: <https://arxiv.org/abs/2308.06954>
- [6] G. Devin DL, “Practical Guide to Data Augmentation for CNN Model Training,” [Online]. Available: <https://medium.com/@tubelwj/practical-guide-to-data-augmentation-for-cnn-model-training-5b7cc9baeed1>

## Appendix

### Results Plots



(a) Training Loss vs Epochs



(b) MIoU vs Epochs